

codeHive

Binary Trees

THE POWER OF TWO CHILDREN

Linear vs. Hierarchical structures: Module 25

codeHive

1. The Binary Constraint

A **Binary Tree** is a hierarchical data structure where each node is restricted to a maximum of **two children**: a Left Child and a Right Child.

Why the Limit?

- **Efficiency**: Balanced binary trees allow for $O(\log n)$ search speeds.
- **Memory**: Can be represented efficiently as simple arrays.
- **Simplicity**: Core algorithms (Traversal, Insertion, Deletion) become standardized.



2. Classifications

Balanced

Height difference between left & right is at most 1.

Full

Every node has either 0 or 2 children.

Complete

All levels filled except possibly the last (filled left-to-right).

Perfect

All internal nodes have 2 children; leaves are on the same level.

3. Implementation

A node in a binary tree is similar to a linked list node, but with `left` and `right` pointers instead of a single `next` pointer.

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Manual Implementation
root = TreeNode('R')
root.left = TreeNode('A')
root.right = TreeNode('B')

# Accessing deeper levels
# root.right.left.data
```

Tree Traversal Methods

There are two primary ways to visit every node:

- **Breadth-First Search (BFS):** Level-by-level exploration.
- **Depth-First Search (DFS):** Exploring the full depth of a branch first (Pre-order, In-order, Post-order).

4. Binary Trees vs. Linear Structures

Operation	Array	Binary Tree
Access	$O(1)$	$O(\log n)$
Insert/Delete	$O(n)$ (Shifting)	$O(\log n)$ (No Shift)

© 2026 codeHive Academy • Foundations of Binary Logic

codeHive